
```

00001 /*
00002 * blinky_rtos/linker.ld -- Linker script for ThreadX-based multi-LED breathe.
00003 *
00004 * RX72N memory map (R5F572NNHxFB):
00005 *   Internal RAM : 0x00000004 - 0x0007FFFF (512 KB, first 4 bytes reserved)
00006 *   Internal ROM : 0xFFE00000 - 0xFFFFFFFF (2 MB)
00007 *
00008 * Fixed hardware vector locations:
00009 *   0xFFFFFFFF80 - 0xFFFFFFFFFC Exception vector table (EXTB)
00010 *   0xFFFFFFFFFC - 0xFFFFFFFFFF Reset vector (points to startup entry)
00011 *
00012 * INTB is loaded by startup.S to point at __rVectors_start (in ROM).
00013 *
00014 * Renesas-syntax sections from ThreadX RXv3 port (tx_timer_interrupt.S etc.):
00015 *   P, P -- code sections (.SECTION P, CODE)
00016 *   B, B -- BSS sections (.SECTION B, DATA)
00017 *   D, D -- initialised data sections
00018 */
00019
00020 MEMORY
00021 {
00022     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC /* 512 KB - 4 bytes */
00023     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000 /* 2 MB */
00024 }
00025
00026 SECTIONS
00027 {
00028     /* Code -- explicit ROM base so section VMA is anchored correctly.
00029     * *(P) captures Renesas-assembly code sections from ThreadX port files. */
00030     .text 0xFFE00000 : AT(0xFFE00000)
00031     {
00032         *(.text*)
00033         *(P)
00034         *(P)
00035         *(.rodata*)
00036         *(C)
00037         *(C)
00038         *(C)
00039         etext = .;
00040     } > ROM
00041
00042     /*
00043     * Relocatable vector table -- startup.S loads INTB with this address.
00044     * Must be 4-byte aligned; 32 entries = 128 bytes.
00045     * No AT() needed: follows .text in ROM, VMA and LMA are the same.
00046     */
00047     .rVectors ALIGN(4) :
00048     {
00049         __rVectors_start = .;
00050         KEEP(*(__rVectors))
00051         __rVectors_end = .;
00052     } > ROM
00053
00054     /*
00055     * Initialised data -- LMA stays in ROM (image stored here), VMA in RAM.
00056     * startup.S copies from __mdata (ROM) to __data..__edata (RAM) on boot.

```

```

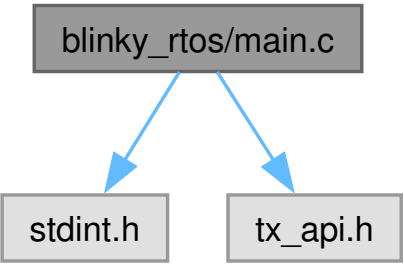
00057  * Required for ThreadX: __tx_thread_system_state starts as 0xF0F0F0F0.
00058  */
00059  __mdata = .;
00060  .data : AT(__mdata)
00061  {
00062      __data = .;
00063      *(.data*)
00064      *(D)
00065      *(D)
00066      *(D)
00067      __edata = .;
00068  } > RAM
00069
00070  /* BSS -- ThreadX needs this zeroed; startup.S handles it.
00071  * B/B/B are Renesas-assembler BSS sections from ThreadX port files. */
00072  .bss :
00073  {
00074      __bss = .;
00075      *(.bss*)
00076      *(COMMON)
00077      *(B)
00078      *(B)
00079      *(B)
00080      __ebss = .;
00081      . = ALIGN(4);
00082      __end = .; /* ThreadX tx_initialize_low_level reads __end */
00083  } > RAM AT>RAM
00084
00085  /*
00086  * Stacks -- USP grows down from top of RAM, ISP 2 KB below that.
00087  * ThreadX uses the interrupt stack pointer (ISP) for ISR execution.
00088  */
00089  __ustack = ORIGIN(RAM) + LENGTH(RAM);
00090  __istack = __ustack - 0x800;
00091
00092  /*
00093  * Exception vector table (EXTB) at fixed hardware address 0xFFFFFFF80.
00094  */
00095  .exvectors 0xFFFFFFF80 : AT(0xFFFFFFF80)
00096  {
00097      LONG(0xFFFFFFF); /* BUSERR */
00098      LONG(0xFFFFFFF); /* reserved */
00099      LONG(0xFFFFFFF); /* FIFERR */
00100      LONG(0xFFFFFFF); /* FRDYI */
00101      LONG(0xFFFFFFF); /* reserved */
00102      LONG(0xFFFFFFF);
00103      LONG(0xFFFFFFF);
00104      LONG(0xFFFFFFF);
00105      LONG(0xFFFFFFF);
00106      LONG(0xFFFFFFF);
00107      LONG(0xFFFFFFF);
00108      LONG(0xFFFFFFF);
00109      LONG(0xFFFFFFF);
00110      LONG(0xFFFFFFF);
00111      LONG(0xFFFFFFF);
00112      LONG(0xFFFFFFF);
00113      LONG(0xFFFFFFF);
00114      LONG(0xFFFFFFF);
00115      LONG(0xFFFFFFF);
00116      LONG(0xFFFFFFF);
00117      LONG(0xFFFFFFF);
00118      LONG(0xFFFFFFF);
00119      LONG(0xFFFFFFF);
00120      LONG(0xFFFFFFF);
00121      LONG(0xFFFFFFF);
00122      LONG(0xFFFFFFF);
00123      LONG(0xFFFFFFF);
00124      LONG(0xFFFFFFF);
00125      LONG(0xFFFFFFF);
00126      LONG(0xFFFFFFF);
00127      LONG(0xFFFFFFF);
00128  } > ROM
00129
00130  /* Fixed reset vector at 0xFFFFFFF */
00131  .fvectors 0xFFFFFFF : AT(0xFFFFFFF)
00132  {
00133      KEEP(*(.fvectors))
00134  } > ROM
00135  }

```

```

#include <stdint.h>
#include "tx_api.h"

```



<<
<<<<
<<<<<<

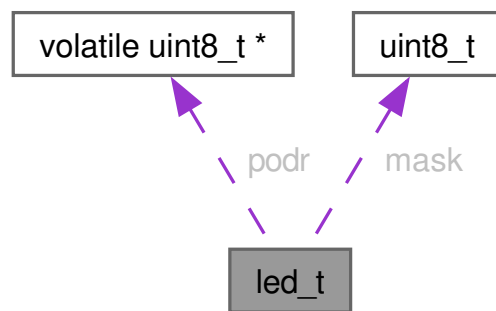
*
*
*
*
*
*
*
*
*
*
*
*
*
*

*
*

*

*

>



*		

	2
	2

```

00106      : uint32_t {
00107      k_max_bright    = 50U,    /**< PWM duty-cycle upper bound */
00108      k_reps_per_step  = 5U,    /**< PWM cycles per brightness step */
00109      k_flash_count    = 3U,    /**< Wave passes at cycle end */
00110      k_flash_on_iters = 4000U, /**< Wave LED ON busy-wait iters (~200 ms at LOCO 240 kHz) */
00111      k_flash_off_iters = 2000U, /**< Wave LED OFF gap busy-wait iters (~100 ms at LOCO 240 kHz) */
00112 } breathe_params_t;

```

```
enum cmt0_cfg_t : uint16_t
```


```

00087         : uint16_t {
00088     k_cmt0_cmcpr_val  = 299U, /**< Compare match for 100 Hz at LOCO/8 */
00089     k_cmt0_cmcr_val  = 0x40U, /**< CMIE=1, CKS=00 (PCLKB/8) */
00090     k_cmtstr0_ch0_bit = 0x01U, /**< CMSTR0 bit 0 = CMT0 start/stop */
00091 } cmt0_cfg_t;

```

```
enum hw_addr_t : uintptr_t
```

[illegible]

```

00042         : uintptr_t {
00043     /* GPIO */
00044     k_port7_pdr_addr = 0x0008C007U, /**< PORT7 direction register */
00045     k_port7_podr_addr = 0x0008C027U, /**< PORT7 output data register */
00046     k_porta_pdr_addr = 0x0008C00AU, /**< PORTA direction register */
00047     k_porta_podr_addr = 0x0008C02AU, /**< PORTA output data register */
00048     k_portb_pdr_addr = 0x0008C00BU, /**< PORTB direction register */
00049     k_portb_podr_addr = 0x0008C02BU, /**< PORTB output data register */
00050
00051     /* CMT0 (compare match timer 0) */
00052     k_cmstr0_addr = 0x00088000U, /**< CMT start register (shared CH0/CH1) */
00053     k_cmt0_cmcr = 0x00088002U, /**< CMT0 control register */
00054     k_cmt0_cmcnt = 0x00088004U, /**< CMT0 counter */
00055     k_cmt0_cmcrr = 0x00088006U, /**< CMT0 compare match register */
00056
00057     /* ICU (interrupt controller) */
00058     k_icu_ir_base = 0x00087000U, /**< IR[] base (IR[n] = base + n) */
00059     k_icu_ier_base = 0x00087200U, /**< IER[] base (IER[n/8] bit n%8) */
00060     k_icu_ipr_base = 0x00087300U, /**< IPR[] base (IPR[n] = base + n) */
00061 } hw_addr_t;

```

```
enum icu_cmt0_cfg_t : uint8_t
```


```

00093         : uint8_t {
00094     k_vect_cmt0    = 28U, /**< CMT0 CMI0 interrupt vector number */
00095     k_cmt0_priority = 3U,  /**< ICU interrupt priority for CMT0 */
00096     k_icu_ier_cmt0  = 3U,  /**< IER register index for vector 28 (28/8) */
00097     k_icu_ier_cmt0_bit = 4U, /**< Bit position within IER[3] (28%8) */
00098 } icu_cmt0_cfg_t;

```

```
enum port7_pins_t : uint8_t
```


```

00070         : uint8_t {
00071     k_pin_p71 = (uint8_t)(1U « 1U), /**< PORT7 pin 1 */
00072     k_pin_p72 = (uint8_t)(1U « 2U), /**< PORT7 pin 2 */
00073 } port7_pins_t;

```

enum [porta_pins_t](#) : uint8_t

--	--

```
00066         : uint8_t {
00067   k_pin_pa7 = (uint8_t)(1U « 7U), /**< PORTA pin 7 */
00068 } porta_pins_t;
```

enum [portb_pins_t](#) : uint8_t


```
00075         : uint8_t {
00076   k_pin_pb0 = (uint8_t)(1U « 0U), /**< PORTB pin 0 */
00077   k_pin_pb1 = (uint8_t)(1U « 1U), /**< PORTB pin 1 */
00078   k_pin_pb2 = (uint8_t)(1U « 2U), /**< PORTB pin 2 */
00079 } portb_pins_t;
```

enum [rtos_prio_t](#) : uint8_t


```
00117         : uint8_t {
00118   k_num_leds      = 6U, /**< LEDs in the breathing sequence */
00119   k_breathe_priority = 10U, /**< breathe_task ThreadX priority */
00120   k_flash_priority  = 9U, /**< flash_task ThreadX priority (runs first) */
00121 } rtos_prio_t;
```

enum [rtos_stack_t](#) : uint16_t

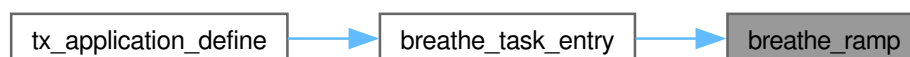
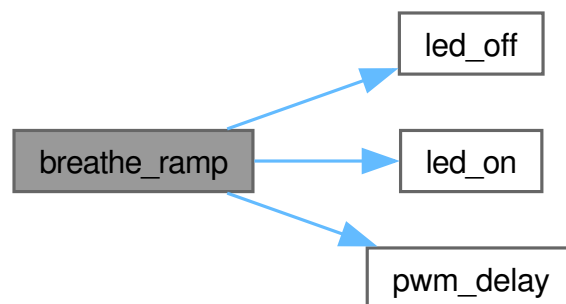

```
00123         : uint16_t {
00124   k_breathe_stack_sz = 512U, /**< breathe_task stack in bytes */
00125   k_flash_stack_sz   = 256U, /**< flash_task stack in bytes */
00126 } rtos_stack_t;
```

```
void _tx_timer_interrupt (  
    void ) [extern]
```



```
void breathe_ramp (  
    const led_t * led,  
    uint32_t invert) [static]
```

```
00269 {  
00270     uint32_t b;  
00271     uint32_t rep;  
00272     uint32_t on_time;  
00273     uint32_t off_time;  
00274  
00275     for (b = 0U; b <= k_max_bright; b++) {  
00276         on_time = (invert == 0U) ? b : (k_max_bright - b);  
00277         off_time = k_max_bright - on_time;  
00278         for (rep = 0U; rep < k_reps_per_step; rep++) {  
00279             if (on_time > 0U) {  
00280                 led_on(led);  
00281                 pwm_delay(on_time);  
00282             }  
00283             led_off(led);  
00284             if (off_time > 0U) {  
00285                 pwm_delay(off_time);  
00286             }  
00287         }  
00288     }  
00289 }
```

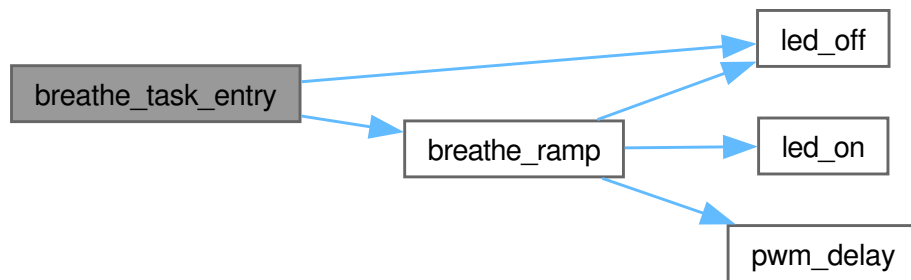


```

void breathe_task_entry (
    ULONG arg) [static]

00295 {
00296     (void)arg;
00297
00298     for (;;) {
00299         uint8_t i;
00300         for (i = 0U; i < k_num_leds; i++) {
00301             breathe_ramp(&s_leds[i], 0U); /* fade in */
00302             breathe_ramp(&s_leds[i], 1U); /* fade out */
00303             led_off(&s_leds[i]);
00304         }
00305
00306         /* Signal flash_task that one full cycle is complete */
00307         (void)tx_semaphore_put(&s_cycle_done_sem);
00308
00309         /* Wait for flash_task to finish before starting the next cycle */
00310         (void)tx_semaphore_get(&s_flash_done_sem, TX_WAIT_FOREVER);
00311     }
00312 }

```

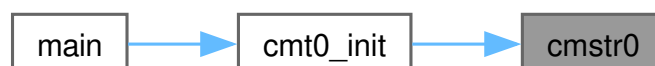


```

volatile uint16_t * cmstr0 (
    void ) [inline], [static]

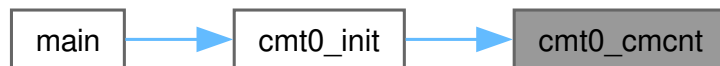
00179 {
00180     return (volatile uint16_t *)k_cmstr0_addr;
00181 }

```



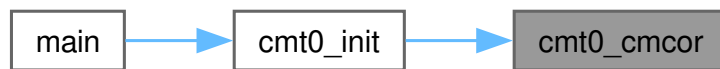
```
volatile uint16_t * cmt0_cmcnt (  
    void )    [inline], [static]
```

```
00187 {  
00188     return (volatile uint16_t *)k_cmt0_cmcnt;  
00189 }
```



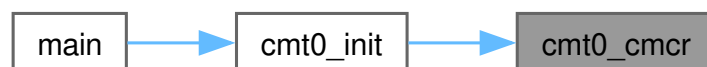
```
volatile uint16_t * cmt0_cmcor (  
    void )    [inline], [static]
```

```
00191 {  
00192     return (volatile uint16_t *)k_cmt0_cmcor;  
00193 }
```



```
volatile uint16_t * cmt0_cmcr (  
    void )    [inline], [static]
```

```
00183 {  
00184     return (volatile uint16_t *)k_cmt0_cmcr;  
00185 }
```

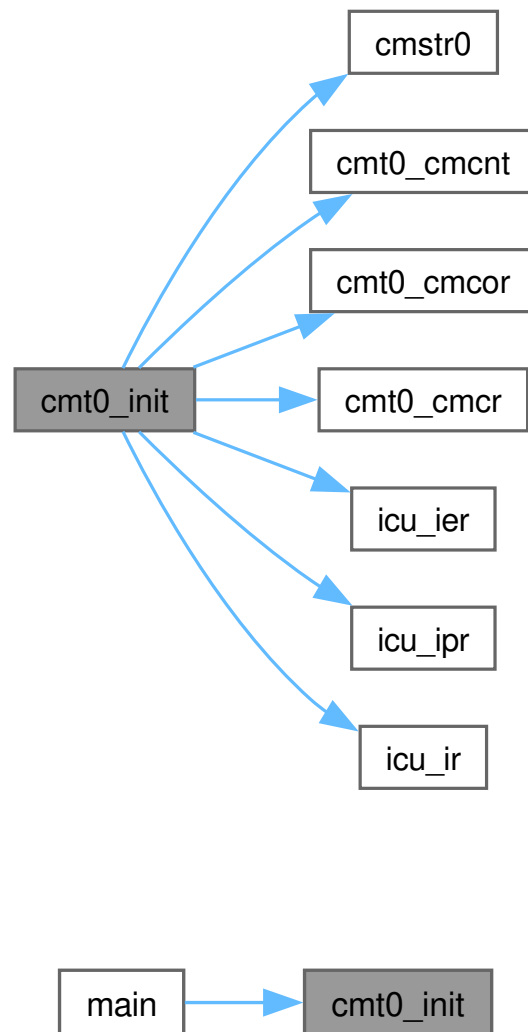


```

void cmt0_init (
    void ) [static]

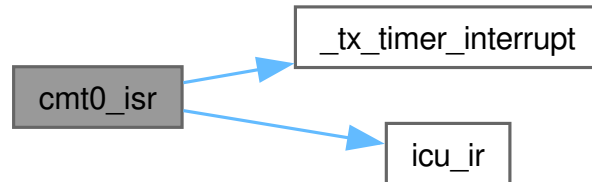
00226 {
00227     *cmstr0() &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit; /* Stop CMT0 */
00228     *cmt0_cmcr() = k_cmt0_cmcr_val; /* PCLKB/8, IRQ enabled */
00229     *cmt0_cmcrr() = k_cmt0_cmcrr_val; /* 100 Hz */
00230     *cmt0_cmcnt() = 0U; /* Reset counter */
00231
00232     *icu_ir(k_vect_cmt0) = 0U; /* Clear pending IRQ */
00233     *icu_ipr(k_vect_cmt0) = k_cmt0_priority; /* Set priority */
00234     *icu_ier(k_icu_ier_cmt0) |= (uint8_t)(1U « k_icu_ier_cmt0_bit); /* Enable */
00235
00236     *cmstr0() |= k_cmstr0_ch0_bit; /* Start CMT0 */
00237     __asm__ volatile("setpsw i"); /* Global interrupt enable */
00238 }

```



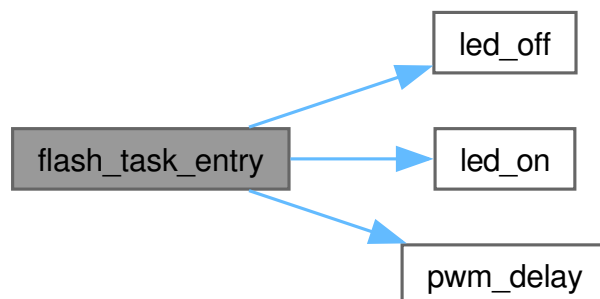
```
void cmt0_isr (
    void )
```

```
00217 {
00218     *icu_ir(k_vect_cmt0) = 0U; /* Clear interrupt request flag */
00219     _tx_timer_interrupt();      /* Drive ThreadX system clock */
00220 }
```



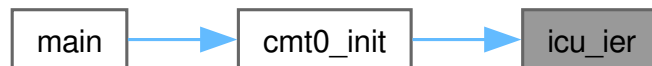
```
void flash_task_entry (
    ULONG arg) [static]
```

```
00318 {
00319     (void)arg;
00320
00321     for (;;) {
00322         uint32_t f;
00323         uint8_t i;
00324
00325         /* Wait for breathe_task to complete one full cycle */
00326         (void)tx_semaphore_get(&s_cycle_done_sem, TX_WAIT_FOREVER);
00327
00328         /* Wave: light each LED one at a time in sequence, k_flash_count passes.
00329          * breathe_task is blocked on s_flash_done_sem during this window,
00330          * so busy-waiting here is safe and avoids any ThreadX sleep dependency. */
00331         for (f = 0U; f < k_flash_count; f++) {
00332             for (i = 0U; i < k_num_leds; i++) {
00333                 led_on(&s_leds[i]);
00334                 pwm_delay(k_flash_on_iters);
00335                 led_off(&s_leds[i]);
00336                 pwm_delay(k_flash_off_iters);
00337             }
00338         }
00339
00340         /* Tell breathe_task it can start the next cycle */
00341         (void)tx_semaphore_put(&s_flash_done_sem);
00342     }
00343 }
```



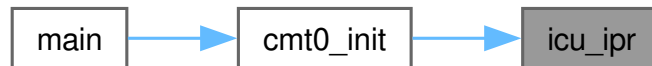
```
volatile uint8_t * icu_ier (  
    uint8_t idx)  [inline], [static]
```

```
00203 {  
00204     return (volatile uint8_t *) (k_icu_ier_base + idx);  
00205 }
```



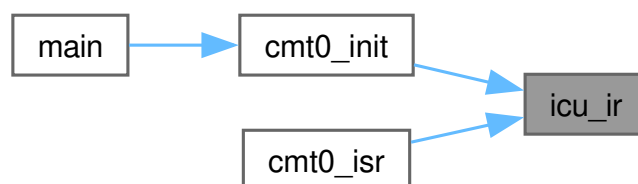
```
volatile uint8_t * icu_ipr (  
    uint8_t vect)  [inline], [static]
```

```
00207 {  
00208     return (volatile uint8_t *) (k_icu_ipr_base + vect);  
00209 }
```



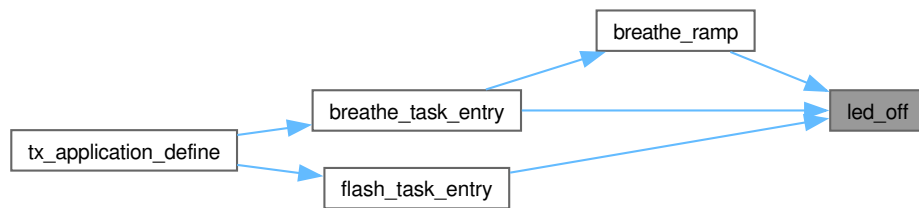
```
volatile uint8_t * icu_ir (  
    uint8_t vect)  [inline], [static]
```

```
00199 {  
00200     return (volatile uint8_t *) (k_icu_ir_base + vect);  
00201 }
```



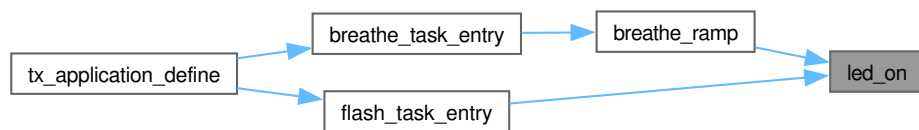
```
void led_off (
    const led_t * led)  [inline], [static]
```

```
00249 {
00250     *led->podr &= (uint8_t)~led->mask;
00251 }
```



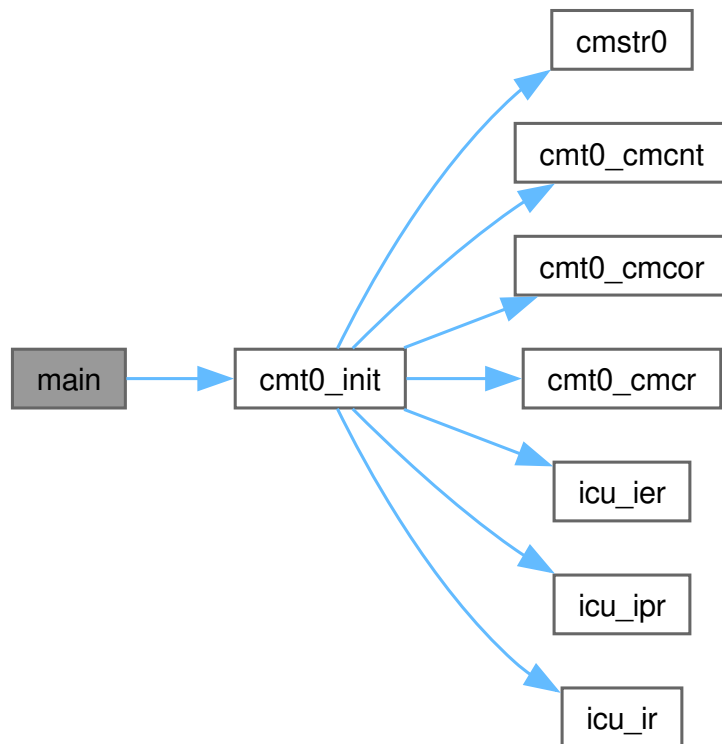
```
void led_on (
    const led_t * led)  [inline], [static]
```

```
00244 {
00245     *led->podr |= led->mask;
00246 }
```



```
int main (
    void )
```

```
00390 {
00391     cmt0_init();
00392     tx_kernel_enter(); /* Never returns */
00393     return 0;
00394 }
```



```
volatile uint8_t * port7_pdr (  
    void ) [inline], [static]
```

```
00151 {  
00152     return (volatile uint8_t *)k_port7_pdr_addr;  
00153 }
```



```
volatile uint8_t * port7_podr (  
    void ) [inline], [static]
```

```
00155 {  
00156     return (volatile uint8_t *)k_port7_podr_addr;  
00157 }
```



```
volatile uint8_t * porta_pdr (  
    void )    [inline], [static]
```

```
00159 {  
00160     return (volatile uint8_t *)k_porta_pdr_addr;  
00161 }
```



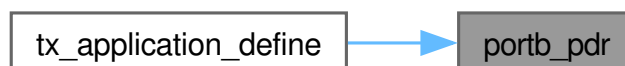
```
volatile uint8_t * porta_podr (  
    void )    [inline], [static]
```

```
00163 {  
00164     return (volatile uint8_t *)k_porta_podr_addr;  
00165 }
```



```
volatile uint8_t * portb_pdr (  
    void )    [inline], [static]
```

```
00167 {  
00168     return (volatile uint8_t *)k_portb_pdr_addr;  
00169 }
```



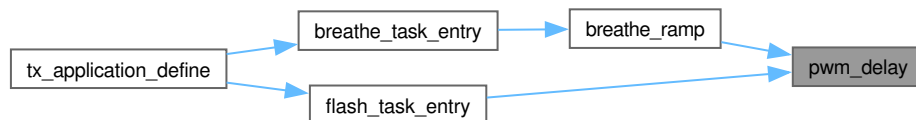
```
volatile uint8_t * portb_podr (
    void )    [inline], [static]
```

```
00171 {
00172     return (volatile uint8_t *)k_portb_podr_addr;
00173 }
```



```
void pwm_delay (
    uint32_t count)    [static]
```

```
00258 {
00259     volatile uint32_t i;
00260     for (i = 0U; i < count; i++) {
00261         __asm__ __volatile__("nop");
00262     }
00263 }
```



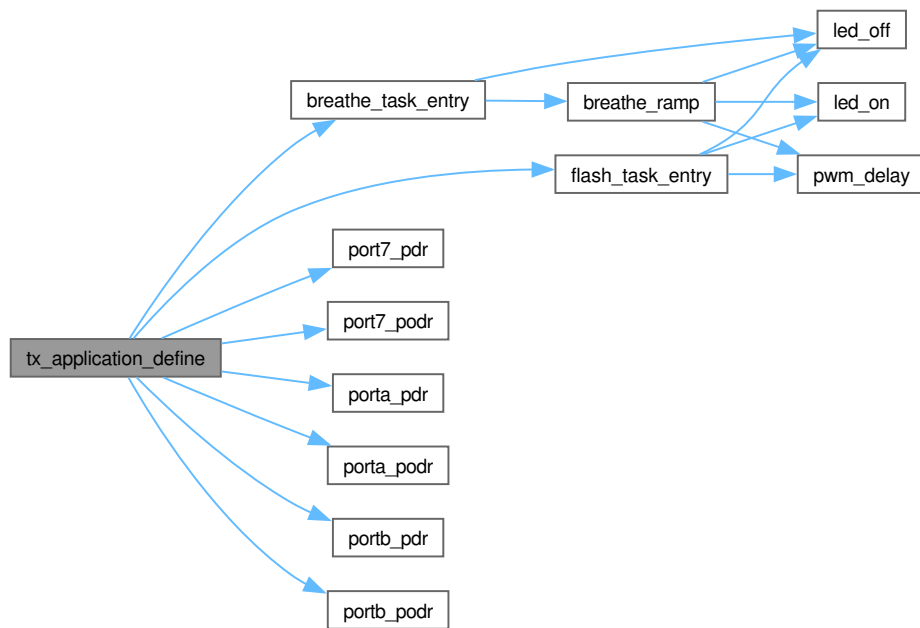
```
void tx_application_define (
    void * first_unused_memory)
```

```
00350 {
00351     (void)first_unused_memory;
00352
00353     /* Semaphores: initial count 0 (both tasks wait until signalled) */
00354     (void)tx_semaphore_create(&s_cycle_done_sem, "cycle_done", 0U);
00355     (void)tx_semaphore_create(&s_flash_done_sem, "flash_done", 1U);
00356
00357     /* Configure LED pins as outputs (PDR bit = 1).
00358      * After reset PMR = 0x00 (all GPIO), no MPC unlock needed. */
00359     *porta_pdr() |= (uint8_t)k_pin_pa7;
00360     *port7_pdr() |= (uint8_t)(k_pin_p71 | k_pin_p72);
00361     *portb_pdr() |= (uint8_t)(k_pin_pb0 | k_pin_pb1 | k_pin_pb2);
00362
00363     /* Build LED table -- order defines breathing sequence */
00364     s_leds[0].pdr = porta_podr(); s_leds[0].mask = (uint8_t)k_pin_pa7;
00365     s_leds[1].pdr = portb_podr(); s_leds[1].mask = (uint8_t)k_pin_pb0;
00366     s_leds[2].pdr = port7_podr(); s_leds[2].mask = (uint8_t)k_pin_p71;
00367     s_leds[3].pdr = port7_podr(); s_leds[3].mask = (uint8_t)k_pin_p72;
00368     s_leds[4].pdr = portb_podr(); s_leds[4].mask = (uint8_t)k_pin_pb1;
00369     s_leds[5].pdr = portb_podr(); s_leds[5].mask = (uint8_t)k_pin_pb2;
00370
00371     /* Create breathe_task */
00372     (void)tx_thread_create(&s_breathe_tcb, "breathe",
00373         breathe_task_entry, 0U,
```

```

00374         s_breathe_stack, k_breathe_stack_sz,
00375         k_breathe_priority, k_breathe_priority,
00376         TX_NO_TIME_SLICE, TX_AUTO_START);
00377
00378     /* Create flash_task */
00379     (void)tx_thread_create(&s_flash_tcb, "flash",
00380         flash_task_entry, 0U,
00381         s_flash_stack, k_flash_stack_sz,
00382         k_flash_priority, k_flash_priority,
00383         TX_NO_TIME_SLICE, TX_AUTO_START);
00384 }

```



```
uint8_t s_breathe_stack[k_breathe_stack_sz] [static]
```

```
TX_THREAD s_breathe_tcb [static]
```

```
TX_SEMAPHORE s_cycle_done_sem [static]
```

TX_SEMAPHORE s_flash_done_sem [static]

uint8_t s_flash_stack[k_flash_stack_sz] [static]

TX_THREAD s_flash_tcb [static]

led_t s_leds[k_num_leds] [static]

```
00001 /**
00002  * @file main.c
00003  * @brief RX72N multi-LED breathing cycle using ThreadX RTOS.
00004  *
00005  * @details
00006  * Two ThreadX threads drive the breathing animation:
00007  *
00008  * breathe_task -- cycles through all 6 LEDs sequentially, applying a
00009  *                software-PWM breathing effect to each one in turn.
00010  *                After completing a full cycle it posts a semaphore.
00011  *
00012  * flash_task    -- waits on the semaphore, runs a wave sequence: each LED
00013  *                lights up one at a time in order (k_flash_count passes),
00014  *                then signals breathe_task to start the next cycle.
00015  *
00016  * Threading replaces the single infinite loop of the bare-metal version.
00017  * Busy-wait delays are used for both PWM and wave timing; the 100 Hz tick is
00018  * too coarse for smooth dimming and tx_thread_sleep is not used.
00019  *
00020  * Clock: LOCO default at 240 kHz (OFS1 = 0xFFFFFFFF, factory reset state).
00021  * Tick:  CMT0 at PCLKB/8 = 30 kHz, CMCOR = 299 -> 100 Hz (10 ms/tick).
00022  *
00023  * Port register addresses: RX72N Hardware Manual r01uh0824ej0111, Ch 22.
00024  *   PDR base = 0x0008C000 + port_index
00025  *   PODR base = 0x0008C020 + port_index
00026  *   Port indices: 7=0x07, A=0x0A, B=0x0B
00027  *
00028  * ICU and CMT0 addresses: RX72N Hardware Manual, Ch 13/15.
00029  *
00030  * SPDX-License-Identifier: MIT
00031  * @copyright Copyright (c) 2026 Locked Inc.
00032  */
00033
00034 #include <stdint.h>
00035 #include "tx_api.h"
00036
00037 /**
=====
00038  * Hardware register addresses
00039  * uintptr_t is mandatory: on RX72N target = uint32_t, on x86-64 host =
00040  * uint64_t. Using uint32_t directly silently truncates on the host.
00041  *
=====
00042  */
00042 typedef enum : uintptr_t {
00043     /* GPIO */
00044     k_port7_pdr_addr = 0x0008C007U, /**< PORT7 direction register */
```

```

00045 k_port7_podr_addr = 0x0008C027U, /**< PORT7 output data register */
00046 k_porta_pdr_addr = 0x0008C00AU, /**< PORTA direction register */
00047 k_porta_podr_addr = 0x0008C02AU, /**< PORTA output data register */
00048 k_portb_pdr_addr = 0x0008C00BU, /**< PORTB direction register */
00049 k_portb_podr_addr = 0x0008C02BU, /**< PORTB output data register */
00050
00051 /* CMT0 (compare match timer 0) */
00052 k_cmstr0_addr = 0x00088000U, /**< CMT start register (shared CH0/CH1) */
00053 k_cmt0_cmcr = 0x00088002U, /**< CMT0 control register */
00054 k_cmt0_cmcnt = 0x00088004U, /**< CMT0 counter */
00055 k_cmt0_cmcpr = 0x00088006U, /**< CMT0 compare match register */
00056
00057 /* ICU (interrupt controller) */
00058 k_icu_ir_base = 0x00087000U, /**< IR[] base (IR[n] = base + n) */
00059 k_icu_ier_base = 0x00087200U, /**< IER[] base (IER[n/8] bit n%8) */
00060 k_icu_ipr_base = 0x00087300U, /**< IPR[] base (IPR[n] = base + n) */
00061 } hw_addr_t;
00062
00063 /*
=====
00064 * Pin bit masks (one enum per port to avoid duplicate-value collisions)
00065 *
=====
*/
00066 typedef enum : uint8_t {
00067     k_pin_pa7 = (uint8_t)(1U « 7U), /**< PORTA pin 7 */
00068 } porta_pins_t;
00069
00070 typedef enum : uint8_t {
00071     k_pin_p71 = (uint8_t)(1U « 1U), /**< PORT7 pin 1 */
00072     k_pin_p72 = (uint8_t)(1U « 2U), /**< PORT7 pin 2 */
00073 } port7_pins_t;
00074
00075 typedef enum : uint8_t {
00076     k_pin_pb0 = (uint8_t)(1U « 0U), /**< PORTB pin 0 */
00077     k_pin_pb1 = (uint8_t)(1U « 1U), /**< PORTB pin 1 */
00078     k_pin_pb2 = (uint8_t)(1U « 2U), /**< PORTB pin 2 */
00079 } portb_pins_t;
00080
00081 /*
=====
00082 * CMT0 configuration constants (LOCO = 240 kHz)
00083 * PCLKB/8 = 30 000 Hz timer clock.
00084 * CMCOR = 30000/100 - 1 = 299 -> 100 Hz tick (10 ms/tick).
00085 * CMCR = 0x40 -> CMIE=1 (interrupt enable), CKS=00 (PCLKB/8).
00086 *
=====
*/
00087 typedef enum : uint16_t {
00088     k_cmt0_cmcpr_val = 299U, /**< Compare match for 100 Hz at LOCO/8 */
00089     k_cmt0_cmcr_val = 0x40U, /**< CMIE=1, CKS=00 (PCLKB/8) */
00090     k_cmstr0_ch0_bit = 0x01U, /**< CMSTR0 bit 0 = CMT0 start/stop */
00091 } cmt0_cfg_t;
00092
00093 typedef enum : uint8_t {
00094     k_vect_cmt0 = 28U, /**< CMT0 CMT0 interrupt vector number */
00095     k_cmt0_priority = 3U, /**< ICU interrupt priority for CMT0 */
00096     k_icu_ier_cmt0 = 3U, /**< IER register index for vector 28 (28/8) */
00097     k_icu_ier_cmt0_bit = 4U, /**< Bit position within IER[3] (28%8) */
00098 } icu_cmt0_cfg_t;
00099
00100 /*
=====
00101 * Breathing / flash timing
00102 * Each delay() iteration ~ 50 us at LOCO 240 kHz (same as bare-metal).
00103 * PWM period = k_max_bright iters = 50 * 50 us = 2.5 ms -> 400 Hz PWM.
00104 * tx_thread_sleep() granularity = 10 ms/tick.
00105 *
=====
*/
00106 typedef enum : uint32_t {
00107     k_max_bright = 50U, /**< PWM duty-cycle upper bound */
00108     k_reps_per_step = 5U, /**< PWM cycles per brightness step */
00109     k_flash_count = 3U, /**< Wave passes at cycle end */
00110     k_flash_on_iters = 4000U, /**< Wave LED ON busy-wait iters (~200 ms at LOCO 240 kHz) */
00111     k_flash_off_iters = 2000U, /**< Wave LED OFF gap busy-wait iters (~100 ms at LOCO 240 kHz) */
00112 } breathe_params_t;
00113
00114 /*
=====
00115 * ThreadX objects
00116 *
=====
*/
00117 typedef enum : uint8_t {
00118     k_num_leds = 6U, /**< LEDs in the breathing sequence */
00119     k_breathe_priority = 10U, /**< breathe_task ThreadX priority */

```

```

00120     k_flash_priority  = 9U, /**< flash_task ThreadX priority (runs first) */
00121 } rtos_prio_t;
00122
00123 typedef enum : uint16_t {
00124     k_breathe_stack_sz = 512U, /**< breathe_task stack in bytes */
00125     k_flash_stack_sz   = 256U, /**< flash_task stack in bytes */
00126 } rtos_stack_t;
00127
00128 static TX_THREAD s_breathe_tcb;
00129 static TX_THREAD s_flash_tcb;
00130 static TX_SEMAPHORE s_cycle_done_sem; /**< Posted by breathe_task, waited by flash_task */
00131 static TX_SEMAPHORE s_flash_done_sem; /**< Posted by flash_task, waited by breathe_task */
00132
00133 static uint8_t s_breathe_stack[k_breathe_stack_sz];
00134 static uint8_t s_flash_stack[k_flash_stack_sz];
00135
00136 /*
=====
00137 * LED descriptor
00138 *
=====
*/
00139 typedef struct {
00140     volatile uint8_t *podr; /**< Port output data register */
00141     uint8_t          mask; /**< Bit mask for this LED's pin */
00142 } led_t;
00143
00144 /* Shared LED table (read-only after init in tx_application_define) */
00145 static led_t s_leds[k_num_leds];
00146
00147 /*
=====
00148 * Inline port register accessors
00149 *
=====
*/
00150 static inline volatile uint8_t *port7_pdr(void)
00151 {
00152     return (volatile uint8_t *)k_port7_pdr_addr;
00153 }
00154 static inline volatile uint8_t *port7_podr(void)
00155 {
00156     return (volatile uint8_t *)k_port7_podr_addr;
00157 }
00158 static inline volatile uint8_t *porta_pdr(void)
00159 {
00160     return (volatile uint8_t *)k_porta_pdr_addr;
00161 }
00162 static inline volatile uint8_t *porta_podr(void)
00163 {
00164     return (volatile uint8_t *)k_porta_podr_addr;
00165 }
00166 static inline volatile uint8_t *portb_pdr(void)
00167 {
00168     return (volatile uint8_t *)k_portb_pdr_addr;
00169 }
00170 static inline volatile uint8_t *portb_podr(void)
00171 {
00172     return (volatile uint8_t *)k_portb_podr_addr;
00173 }
00174
00175 /*
=====
00176 * CMT0 register accessors
00177 *
=====
*/
00178 static inline volatile uint16_t *cmstr0(void)
00179 {
00180     return (volatile uint16_t *)k_cmstr0_addr;
00181 }
00182 static inline volatile uint16_t *cmt0_cmcr(void)
00183 {
00184     return (volatile uint16_t *)k_cmt0_cmcr;
00185 }
00186 static inline volatile uint16_t *cmt0_cmcnt(void)
00187 {
00188     return (volatile uint16_t *)k_cmt0_cmcnt;
00189 }
00190 static inline volatile uint16_t *cmt0_cmcor(void)
00191 {
00192     return (volatile uint16_t *)k_cmt0_cmcor;
00193 }
00194
00195 /*
=====
00196 * ICU register accessors

```

```

00197 *
=====
00198 */
00199 static inline volatile uint8_t *icu_ir(uint8_t vect)
00200 {
00201     return (volatile uint8_t *) (k_icu_ir_base + vect);
00202 }
00203 static inline volatile uint8_t *icu_ier(uint8_t idx)
00204 {
00205     return (volatile uint8_t *) (k_icu_ier_base + idx);
00206 }
00207 static inline volatile uint8_t *icu_ipr(uint8_t vect)
00208 {
00209     return (volatile uint8_t *) (k_icu_ipr_base + vect);
00210 }
00211 /*
=====
00212 * ThreadX tick source -- CMT0 ISR
00213 *
=====
00214 */
00215 extern void _tx_timer_interrupt(void);
00216 void __attribute__((interrupt)) cmt0_isr(void)
00217 {
00218     icu_ir(k_vect_cmt0) = 0U; /* Clear interrupt request flag */
00219     _tx_timer_interrupt(); /* Drive ThreadX system clock */
00220 }
00221
00222 /*
=====
00223 * CMT0 initialisation (called before tx_kernel_enter)
00224 *
=====
00225 */
00226 static void cmt0_init(void)
00227 {
00228     *cmstr0() &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit; /* Stop CMT0 */
00229     *cmt0_cmcr() = k_cmt0_cmcr_val; /* PCLKB/8, IRQ enabled */
00230     *cmt0_cmcrr() = k_cmt0_cmcrr_val; /* 100 Hz */
00231     *cmt0_cmcnt() = 0U; /* Reset counter */
00232
00233     icu_ir(k_vect_cmt0) = 0U; /* Clear pending IRQ */
00234     icu_ipr(k_vect_cmt0) = k_cmt0_priority; /* Set priority */
00235     icu_ier(k_icu_ier_cmt0) |= (uint8_t)(1U << k_icu_ier_cmt0_bit); /* Enable */
00236
00237     *cmstr0() |= k_cmstr0_ch0_bit; /* Start CMT0 */
00238     __asm__ volatile("setpsw i"); /* Global interrupt enable */
00239 }
00240 /*
=====
00241 * LED helpers
00242 *
=====
00243 */
00244 static inline void led_on(const led_t *led)
00245 {
00246     *led->podr |= led->mask;
00247 }
00248 static inline void led_off(const led_t *led)
00249 {
00250     *led->podr &= (uint8_t)~led->mask;
00251 }
00252
00253 /*
=====
00254 * Busy-wait delay (used for sub-tick software PWM)
00255 * One iteration ~ 50 us at LOCO 240 kHz.
00256 *
=====
00257 */
00258 static void pwm_delay(uint32_t count)
00259 {
00260     volatile uint32_t i;
00261     for (i = 0U; i < count; i++) {
00262         __asm__ volatile("nop");
00263     }
00264 }
00265 /*
=====
00266 * Software PWM ramp -- one direction of the breathe animation.
00267 *
=====

```

```

00268 */
00269 static void breathe_ramp(const led_t *led, uint32_t invert)
00270 {
00271     uint32_t b;
00272     uint32_t rep;
00273     uint32_t on_time;
00274     uint32_t off_time;
00275     for (b = 0U; b <= k_max_bright; b++) {
00276         on_time = (invert == 0U) ? b : (k_max_bright - b);
00277         off_time = k_max_bright - on_time;
00278         for (rep = 0U; rep < k_reps_per_step; rep++) {
00279             if (on_time > 0U) {
00280                 led_on(led);
00281                 pwm_delay(on_time);
00282             }
00283             led_off(led);
00284             if (off_time > 0U) {
00285                 pwm_delay(off_time);
00286             }
00287         }
00288     }
00289 }
00290
00291 /*
=====
00292 * breathe_task -- cycles through all LEDs, posts semaphore when done.
00293 *
=====
*/
00294 static void breathe_task_entry(ULONG arg)
00295 {
00296     (void)arg;
00297
00298     for (;;) {
00299         uint8_t i;
00300         for (i = 0U; i < k_num_leds; i++) {
00301             breathe_ramp(&s_leds[i], 0U); /* fade in */
00302             breathe_ramp(&s_leds[i], 1U); /* fade out */
00303             led_off(&s_leds[i]);
00304         }
00305
00306         /* Signal flash_task that one full cycle is complete */
00307         (void)tx_semaphore_put(&s_cycle_done_sem);
00308
00309         /* Wait for flash_task to finish before starting the next cycle */
00310         (void)tx_semaphore_get(&s_flash_done_sem, TX_WAIT_FOREVER);
00311     }
00312 }
00313
00314 /*
=====
00315 * flash_task -- waits for cycle complete, flashes all LEDs, signals back.
00316 *
=====
*/
00317 static void flash_task_entry(ULONG arg)
00318 {
00319     (void)arg;
00320
00321     for (;;) {
00322         uint32_t f;
00323         uint8_t i;
00324
00325         /* Wait for breathe_task to complete one full cycle */
00326         (void)tx_semaphore_get(&s_cycle_done_sem, TX_WAIT_FOREVER);
00327
00328         /* Wave: light each LED one at a time in sequence, k_flash_count passes.
00329          * breathe_task is blocked on s_flash_done_sem during this window,
00330          * so busy-waiting here is safe and avoids any ThreadX sleep dependency. */
00331         for (f = 0U; f < k_flash_count; f++) {
00332             for (i = 0U; i < k_num_leds; i++) {
00333                 led_on(&s_leds[i]);
00334                 pwm_delay(k_flash_on_iters);
00335                 led_off(&s_leds[i]);
00336                 pwm_delay(k_flash_off_iters);
00337             }
00338         }
00339
00340         /* Tell breathe_task it can start the next cycle */
00341         (void)tx_semaphore_put(&s_flash_done_sem);
00342     }
00343 }
00344
00345 /*
=====
00346 * tx_application_define -- called by ThreadX after tx_kernel_enter().

```

```

00347 * Creates semaphores, configures GPIO, and starts both threads.
00348 *
=====
*/
00349 void tx_application_define(void *first_unused_memory)
00350 {
00351     (void)first_unused_memory;
00352
00353     /* Semaphores: initial count 0 (both tasks wait until signalled) */
00354     (void)tx_semaphore_create(&s_cycle_done_sem, "cycle_done", 0U);
00355     (void)tx_semaphore_create(&s_flash_done_sem, "flash_done", 1U);
00356
00357     /* Configure LED pins as outputs (PDR bit = 1).
00358      * After reset PMR = 0x00 (all GPIO), no MPC unlock needed. */
00359     *porta_pdr() |= (uint8_t)k_pin_pa7;
00360     *port7_pdr() |= (uint8_t)(k_pin_p71 | k_pin_p72);
00361     *portb_pdr() |= (uint8_t)(k_pin_pb0 | k_pin_pb1 | k_pin_pb2);
00362
00363     /* Build LED table -- order defines breathing sequence */
00364     s_leds[0].pdr = porta_pdr(); s_leds[0].mask = (uint8_t)k_pin_pa7;
00365     s_leds[1].pdr = portb_pdr(); s_leds[1].mask = (uint8_t)k_pin_pb0;
00366     s_leds[2].pdr = port7_pdr(); s_leds[2].mask = (uint8_t)k_pin_p71;
00367     s_leds[3].pdr = port7_pdr(); s_leds[3].mask = (uint8_t)k_pin_p72;
00368     s_leds[4].pdr = portb_pdr(); s_leds[4].mask = (uint8_t)k_pin_pb1;
00369     s_leds[5].pdr = portb_pdr(); s_leds[5].mask = (uint8_t)k_pin_pb2;
00370
00371     /* Create breathe_task */
00372     (void)tx_thread_create(&s_breathe_tcb, "breathe",
00373         breathe_task_entry, 0U,
00374         s_breathe_stack, k_breathe_stack_sz,
00375         k_breathe_priority, k_breathe_priority,
00376         TX_NO_TIME_SLICE, TX_AUTO_START);
00377
00378     /* Create flash_task */
00379     (void)tx_thread_create(&s_flash_tcb, "flash",
00380         flash_task_entry, 0U,
00381         s_flash_stack, k_flash_stack_sz,
00382         k_flash_priority, k_flash_priority,
00383         TX_NO_TIME_SLICE, TX_AUTO_START);
00384 }
00385
00386 /*
=====
00387 * main -- initialise CMT0 tick source, then hand off to ThreadX forever.
00388 *
=====
*/
00389 int main(void)
00390 {
00391     cmt0_init();
00392     tx_kernel_enter(); /* Never returns */
00393     return 0;
00394 }

```
